

Laboratório de Eng. Elétrica

Grupo 1

2026-08-27

Fase 0: Conceitos básicos e Fundamentos

- ☒ Introdução
- ☒ Circuito LED com bateria de 5v
- ☒ Circuito Blink (Hello World no Arduino IDE)
- ☒ Semáforo
- ☒ Comunicação Serial
- ☒ Conversão “A/D”
- ☒ PWM-Pulse Width Modulation
- ☒ Botão Interruptor (Pull Up e Pull Down)
- ☒ LED RGB

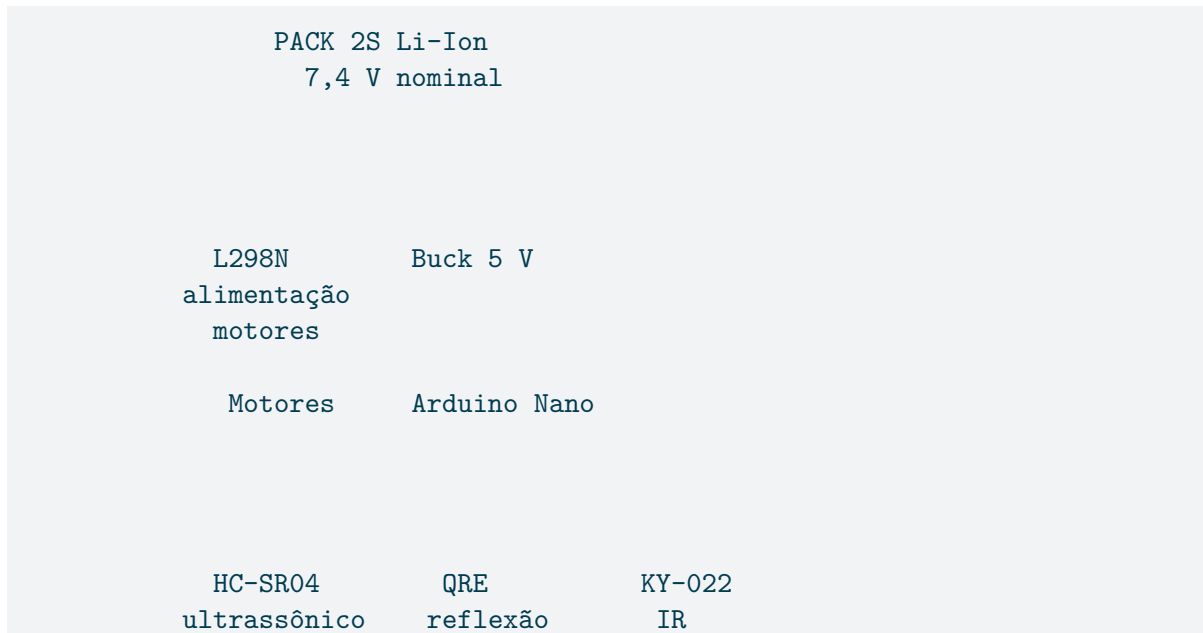
Fase 1: Introdução à Robótica e programação em C++

Componentes que serão utilizados

- Arduino Nano
- Placa de Expansão para Arduino Nano
- Sensor Ultrassônico - HC-SR04
- Sensor de Refletância QRE - Analógico
- Módulo Receptor Infravermelho IR VS1838B KY-022
- Motor DC 3-6V com Caixa de Redução de Aço
- Mini Ponte H Dupla L298N
- Pack de 2 Baterias (2S) Li-Ion 18650 7,4 com Placa BMS.

IMPORTANTE: Mencionem a função do BMS. VL53L0X Sensor de Distância Laser.

Arquitetura geral



Todos os GND precisam estar em comum.

2. Arduino Nano → L298N

Arduino Nano	L298N	Função
D5	ENA	PWM Motor A
D7	IN1	Direção A
D8	IN2	Direção A
D6	ENB	PWM Motor B
D9	IN3	Direção B
D10	IN4	Direção B
GND	GND	Ground

Motores

L298N	Motor
OUT1	Motor A fio 1
OUT2	Motor A fio 2
OUT3	Motor B fio 1
OUT4	Motor B fio 2

Se um motor estiver girando invertido, inverta a polaridade dos fios daquele motor.

3. Alimentação do Arduino Nano

Bateria 7,4 V	Buck 5 V	Nano 5V
		Sensores 5V

Entrada e Saída:

Buck 5V	Nano 5V
Buck GND	Nano GND

E o GND continua comum ao aterramento:

Nano GND	L298N GND
	HC-SR04 GND
	QRE GND
	KY-022 GND

Os sinais PWM e sensores têm a mesma referência elétrica.

Missões (27-08-2026)

Item postado: 25 de jul.

- [x] 1 Gravação de Código - OK

Procurem o exemplo Blink, disponível na IDE do Arduino, e façam a gravação do código no Arduino Nano.

```
// The setup function runs once when you press reset or power the board
void setup() {
  // Initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// The loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // Change state of the LED by setting the pin to the HIGH state
  delay(1000);                      // Wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // Change state of the LED by setting the pin to the LOW state
  delay(1000);                      // Wait for a second
}
```

- [x] 2 Comunicação Serial

Façam um código que envie a mensagem “Hello World!!!” pelo terminal serial a cada 1s. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("Hello world!");
  delay(1000);
}
```

- [x] 3 Sensor Ultrassônico

Façam um código que envie pelo terminal serial a distância lida pelo sensor ultrassônico a cada 1s. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código.

```

// Variáveis
#define trig 2
#define echo 3

void setup() {
    Serial.begin(9600);
    // Configurando os pinos digitais
    pinMode(trig, OUTPUT);
    pinMode(echo, INPUT);
}

void loop() {
    long duracao;
    float distancia;

    // Gerando o pulso da Onda Digital do sensor ultrassônico
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);

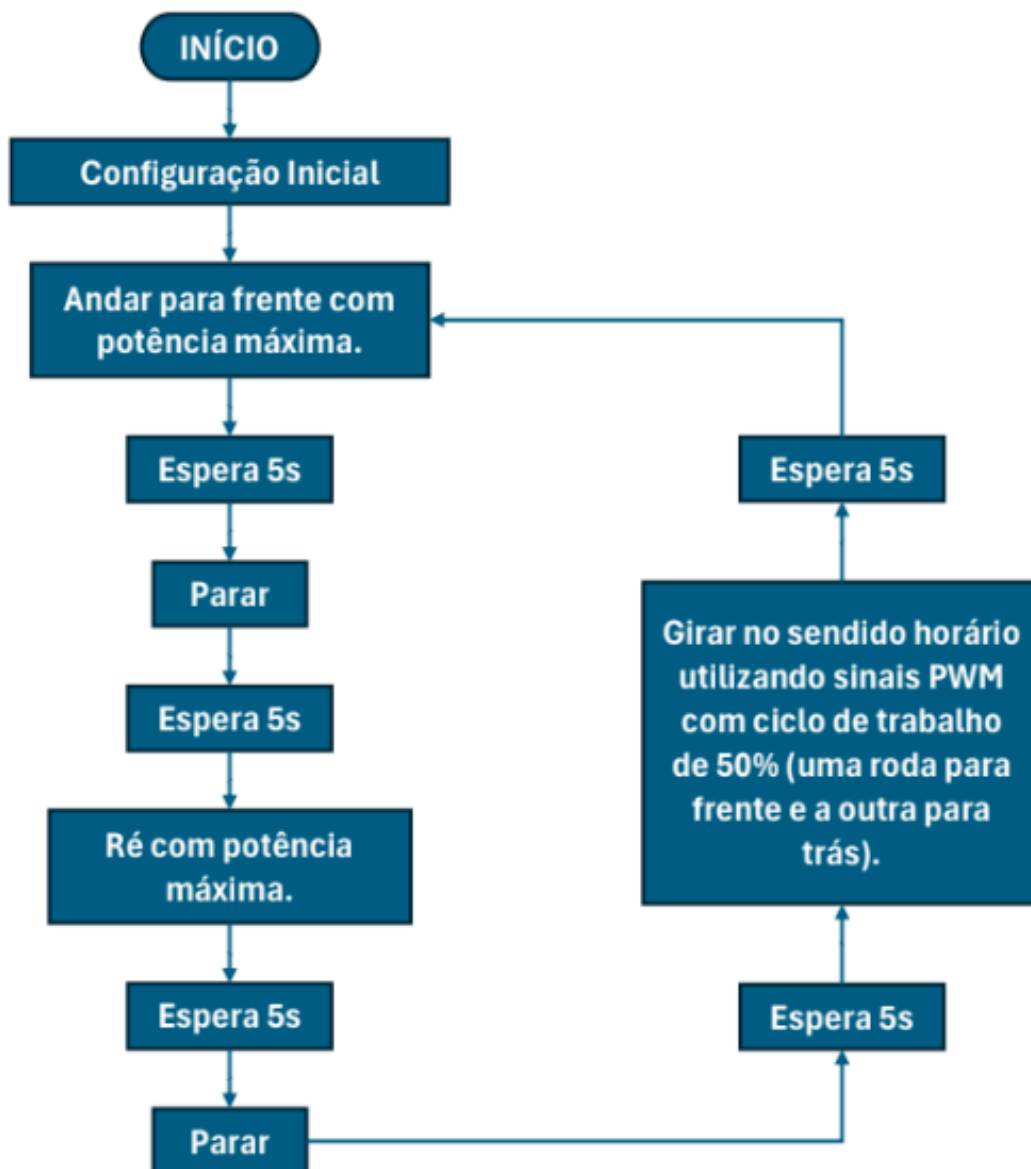
    // Medindo o tempo de retorno
    duracao = pulseIn(echo, HIGH);

    // Calcula a distância em cm
    float vel_som = 0.0343;
    distancia = duracao * vel_som / 2;

    Serial.print("Distancia: ");
    Serial.print(distancia);
    Serial.println(" cm");
    delay(1000);
}

```

- [] 4 Driver e Motores



Façam um código que controle o funcionamento dos motores de acordo com o fluxograma anexado nesta atividade.

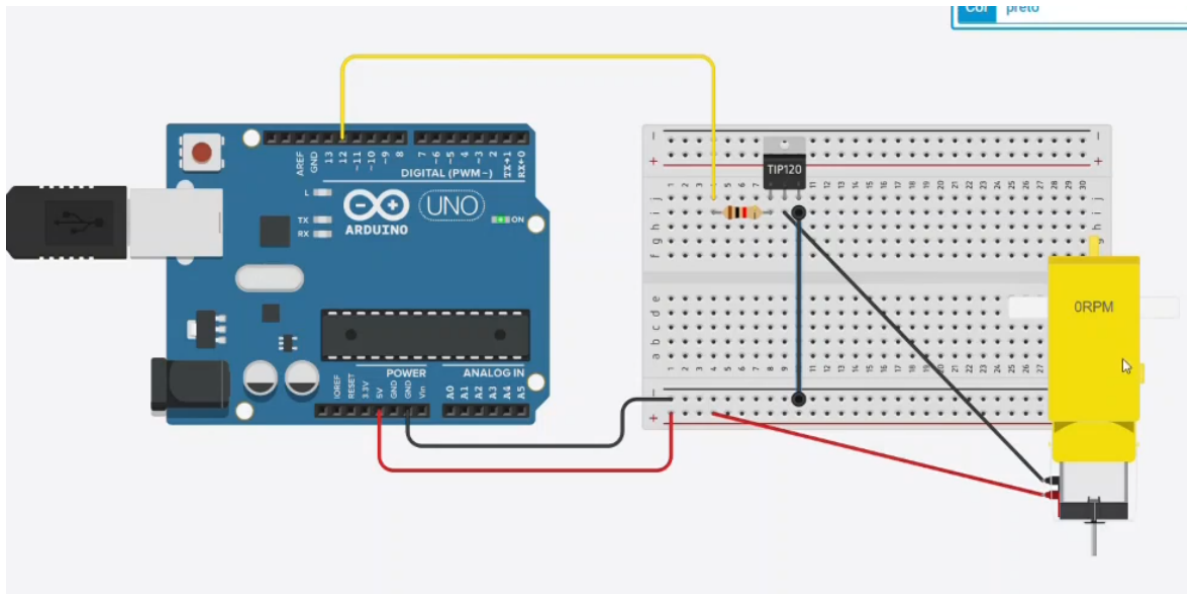


Figure 1: Fluxograma Motor

Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código.

```
/*
 * Arduino Nano + L298N
 * 2 motores DC
 */

// Variáveis e Configurações
const int PWM_max = 255;
const int PWM_50 = 128;
const unsigned long TEMPO_ESPERA = 5000; // 5 segundos

// Motor esquerdo
const int PWM_MOTOR_ESQUERDO = 5;
const int IN1_MOTOR_ESQUERDO = 7;
const int IN2_MOTOR_ESQUERDO = 8;

// Motor direito
const int PWM_MOTOR_DIREITO = 6;
const int IN3_MOTOR_DIREITO = 9;
const int IN4_MOTOR_DIREITO = 10;
```

```

// =====
// FUNÇÕES DOS MOTORES
// =====
void moverFrente(int velocidadePWM) {
    /*
    * MOTOR ESQUERDO
    * Isso cria uma polaridade no motor, considerando a ligação dos fios,
    * faz a roda esquerda girar para frente.
    */
    digitalWrite(IN1_MOTOR_ESQUERDO, HIGH);
    digitalWrite(IN2_MOTOR_ESQUERDO, LOW);

    /*
    * MOTOR DIREITO
    */
    digitalWrite(IN3_MOTOR_DIREITO, HIGH);
    digitalWrite(IN4_MOTOR_DIREITO, LOW);

    /*
    * ENA e ENB recebem PWM.
    * 0 = motor desligado
    * 128 50%
    * 255 = potência máxima
    */
    analogWrite(PWM_MOTOR_ESQUERDO, velocidadePWM);
    analogWrite(PWM_MOTOR_DIREITO, velocidadePWM);
}

void moverRe(int velocidadePWM) {
    /*
    * MOTOR ESQUERDO
    * Invertemos a polaridade:
    * IN1 = LOW
    * IN2 = HIGH
    * Agora a corrente atravessa o motor no sentido oposto ao movimento para frente.
    */
    digitalWrite(IN1_MOTOR_ESQUERDO, LOW);
    digitalWrite(IN2_MOTOR_ESQUERDO, HIGH);

    /*
    * MOTOR DIREITO
    */
}

```



```

digitalWrite(IN3_MOTOR_DIREITO, LOW);
digitalWrite(IN4_MOTOR_DIREITO, HIGH);

// PWM
analogWrite(PWM_MOTOR_ESQUERDO, velocidadePWM);
analogWrite(PWM_MOTOR_DIREITO, velocidadePWM);
}

void pararMotores() {
    // PWM = 0. Retiramos a energia efetiva enviada aos motores.
    analogWrite(PWM_MOTOR_ESQUERDO, 0);
    analogWrite(PWM_MOTOR_DIREITO, 0);

    // Não existe diferença de potencial aplicada entre os terminais do motor.
    // Motor A → parado
    digitalWrite(IN1_MOTOR_ESQUERDO, LOW);
    digitalWrite(IN2_MOTOR_ESQUERDO, LOW);

    // Motor B → parado
    digitalWrite(IN3_MOTOR_DIREITO, LOW);
    digitalWrite(IN4_MOTOR_DIREITO, LOW);
}

void girarSentidoHorario(int velocidadePWM) {
    /*
    * Para girar sobre o próprio eixo,
    * fazemos uma roda andar para frente
    * enquanto a outra anda para trás.
    *
    * MOTOR ESQUERDO → FRENTE
    * IN1 = HIGH
    * IN2 = LOW
    */
    digitalWrite(IN1_MOTOR_ESQUERDO, HIGH);
    digitalWrite(IN2_MOTOR_ESQUERDO, LOW);

    /*
    * MOTOR DIREITO → RÉ
    * IN3 = LOW
    * IN4 = HIGH
    */
    digitalWrite(IN3_MOTOR_DIREITO, LOW);

```

```

digitalWrite(IN4_MOTOR_DIREITO, HIGH);

/*
 * PWM de ambos os motores.
 * velocidadePWM = 128 (aproximadamente 50% do duty cycle).
 */
analogWrite(PWM_MOTOR_ESQUERDO, velocidadePWM);
analogWrite(PWM_MOTOR_DIREITO, velocidadePWM);
}

void setup() {
  // Motor Esquerdo
  pinMode(PWM_MOTOR_ESQUERDO, OUTPUT);
  pinMode(IN1_MOTOR_ESQUERDO, OUTPUT);
  pinMode(IN2_MOTOR_ESQUERDO, OUTPUT);

  // Motor Direito
  pinMode(PWM_MOTOR_DIREITO, OUTPUT);
  pinMode(IN3_MOTOR_DIREITO, OUTPUT);
  pinMode(IN4_MOTOR_DIREITO, OUTPUT);

  // Estado inicial -> parado
  pararMotores();
  delay(1000);
}

void loop() {
  // 1. ANDAR PARA FRENTE - POTÊNCIA MÁXIMA
  moverFrente(PWM_max);
  delay(TEMPO_ESPERA);

  // 2. Parar e esperar 5 segundos
  pararMotores();
  delay(TEMPO_ESPERA);

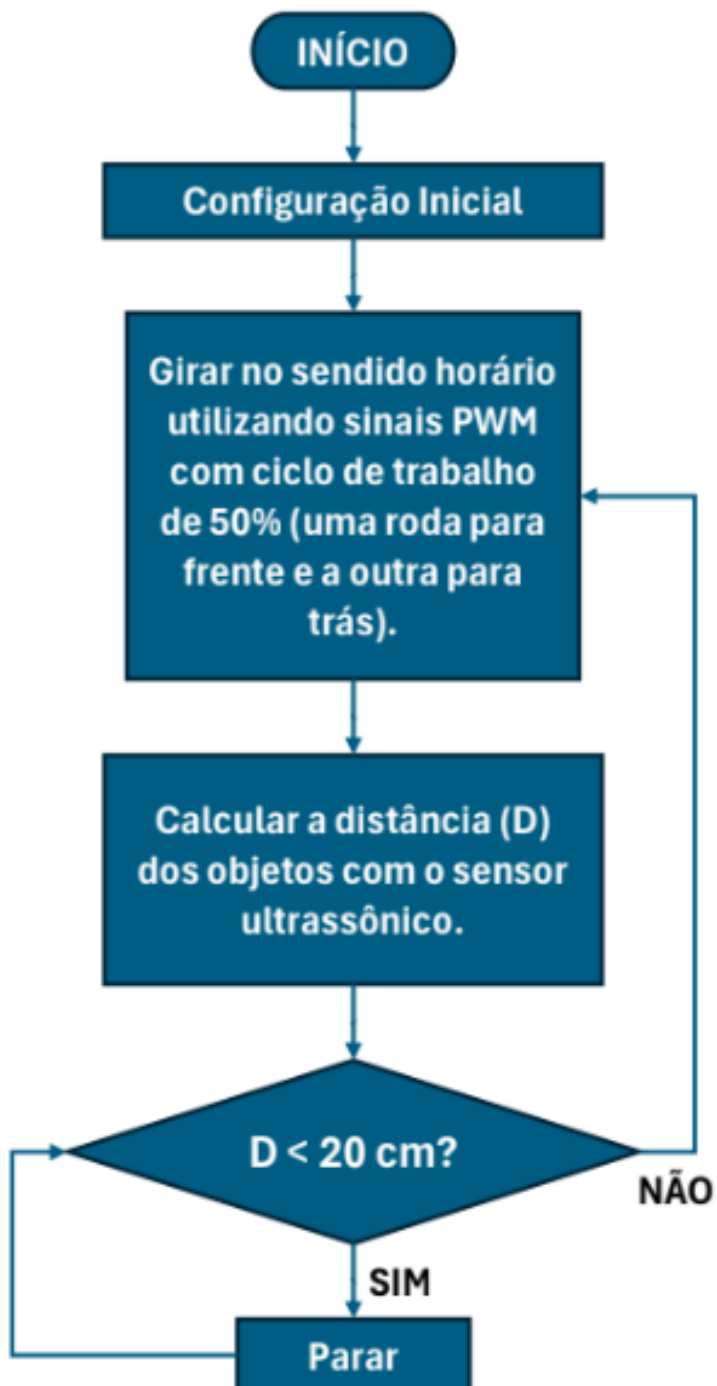
  // 3. RÉ - POTÊNCIA MÁXIMA
  moverRe(PWM_max);
  delay(TEMPO_ESPERA);

  // 4. Parar e esperar 5 segundos
  pararMotores();
  delay(TEMPO_ESPERA);
}

```

```
// 5. GIRAR NO SENTIDO HORÁRIO (PWM = 50%)  
girarSentidoHorario(PWM_50);  
delay(TEMPO_ESPERA);  
}
```


- [] DESAFIO 1 (Modo Radar)

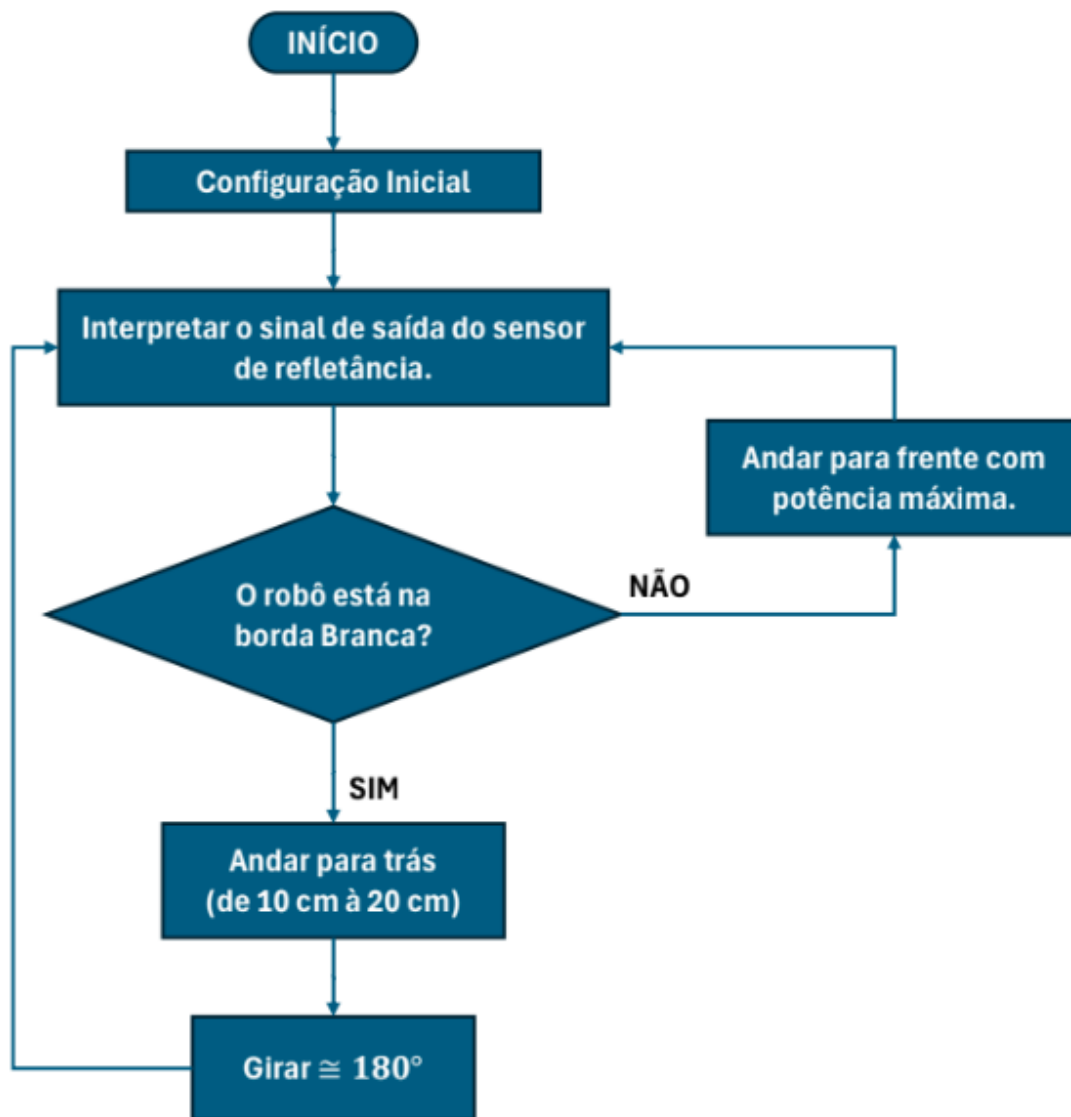


Façam um código que controle o funcionamento dos motores de acordo com o fluxograma anexado nesta atividade. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código. Após o professor anotar na planilha de acompanhamento (Checklist) que a tarefa foi cumprida, 1 integrante de cada equipe (apenas 1) deve fazer a entrega do arquivo do Arduino (*.ino) no Google Classroom. O aluno que entregar a atividade deve indicar (nos comentários) os nomes dos colegas que participaram ativamente do desenvolvimento da tarefa.

- [] 6 Sensor de Refletância

Façam um código que interprete o sinal de saída do sensor de refletância, indicando quanto o robô está na parte preta da arena e quando o mesmo se encontra em cima da borda branca. Envie o resultado da conversão AD (número de 0 à 1023) e o resultado da análise (PRETO ou BRANCO) pela porta serial, atualizando a informação a cada 1s. Exemplo fictício do texto a ser enviado pelo terminal serial: “512 / PRETO”. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código. Após o professor anotar na planilha de acompanhamento (Checklist) que a tarefa foi cumprida, 1 integrante de cada equipe (apenas 1) deve fazer a entrega do arquivo do Arduino (*.ino) no Google Classroom. O aluno que entregar a atividade deve indicar (nos comentários) os nomes dos colegas que participaram ativamente do desenvolvimento da tarefa.

- [] 7 DESAFIO 2 (Vai e Volta)

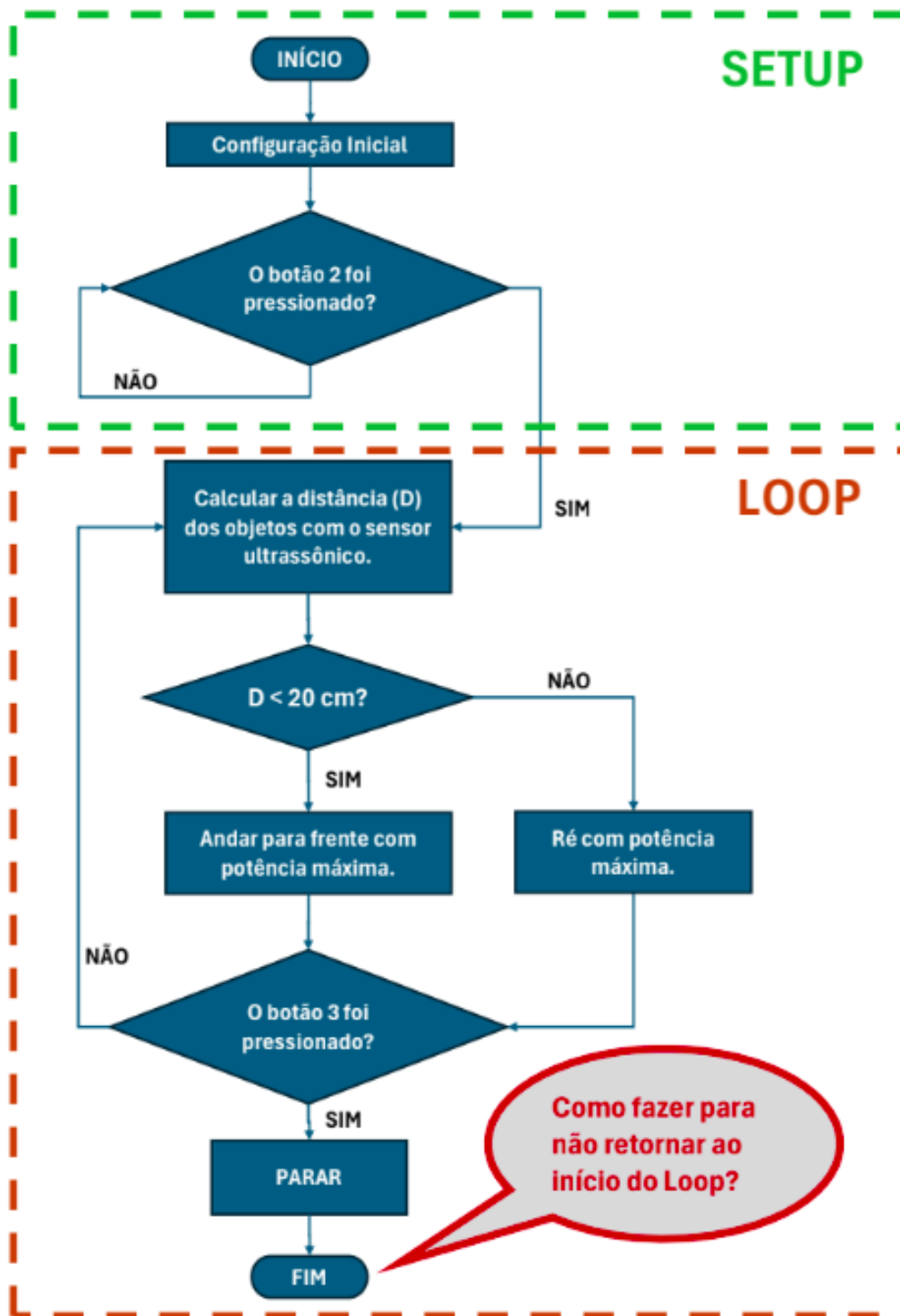


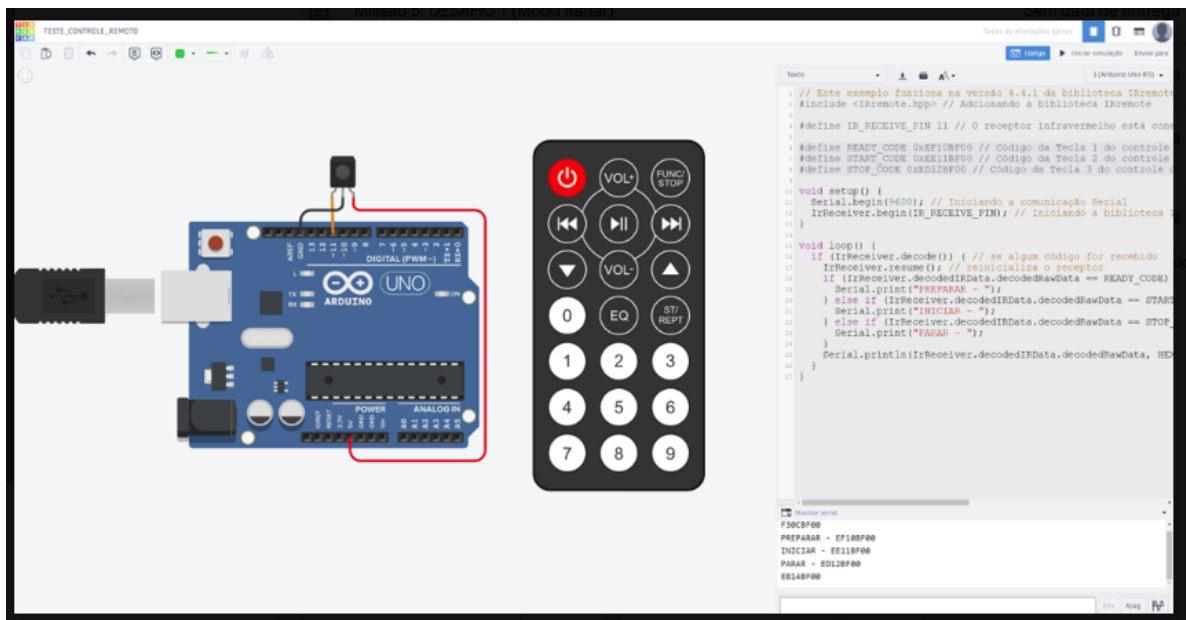
Façam um código que controle o funcionamento dos motores de acordo com o fluxograma anexado nesta atividade. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código. Após o professor anotar na planilha de acompanhamento (Checklist) que a tarefa foi cumprida, 1 integrante de cada equipe (apenas 1) deve fazer a entrega do arquivo do Arduino (*.ino) no Google Classroom. O aluno que entregar a atividade deve indicar (nos comentários) os nomes dos colegas que participaram ativamente do desenvolvimento

da tarefa.

- [] 8 Controle Remoto

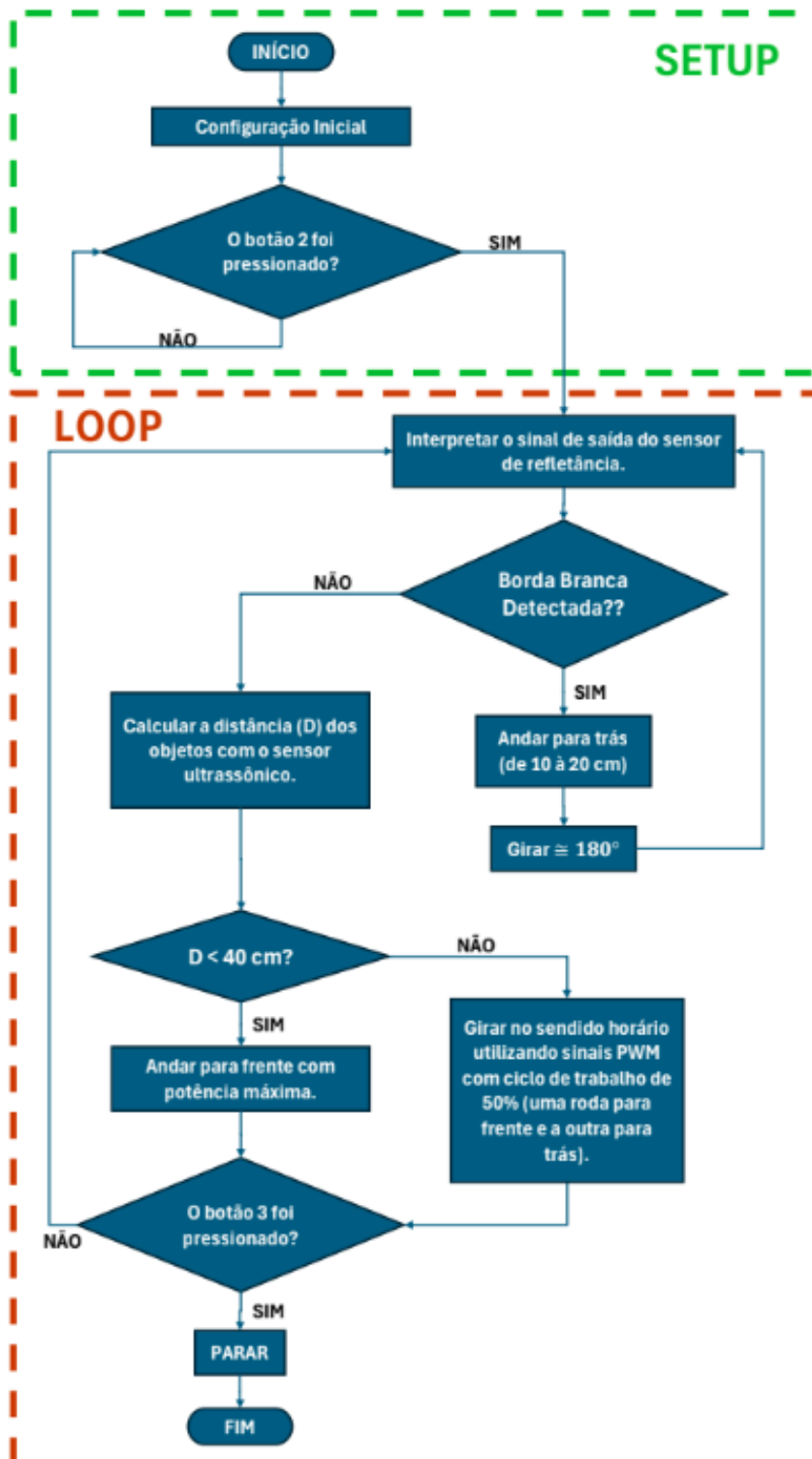
Estudem o código exemplo anexado na atividade e testem seu funcionamento. Façam um código que controle o funcionamento dos motores de acordo com o fluxograma anexado nesta atividade. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código. Após o professor anotar na planilha de acompanhamento (Checklist) que a tarefa foi cumprida, 1 integrante de cada equipe (apenas 1) deve fazer a entrega do arquivo do Arduino (*.ino) no Google Classroom. O aluno que entregar a atividade deve indicar (nos comentários) os nomes dos colegas que participaram ativamente do desenvolvimento da tarefa. DICA: O código exemplo pode ser testado no Tinkercad (ver imagem em anexo). No entanto, as linhas 6, 7 e 8 precisam ser modificadas, visto que o modelo de controle utilizado no Tinkercad é diferente do que usamos em sala. Recomendamos que tentem adiantar esta tarefa em casa, pois ela é a mais difícil até o momento.





- [] 9 DESAFIO 3 (Sumô Básico)

Façam um código que controle o funcionamento dos motores de acordo com o fluxograma anexado nesta atividade. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código. Após o professor anotar na planilha de acompanhamento (Checklist) que a tarefa foi cumprida, 1 integrante de cada equipe (apenas 1) deve fazer a entrega do arquivo do Arduino (*.ino) no Google Classroom. O aluno que entregar a atividade deve indicar (nos comentários) os nomes dos colegas que participaram ativamente do desenvolvimento da tarefa. DICA: Esta missão utiliza conceitos aprendidos nas missões anteriores. Consultem os seus códigos.



- [] 10 Sensores Laterais

- 1) Parabéns! Você chegou na última missão da FASE 2 e desbloqueou um item, o sensor de distância laser VL53L0X. Este sensor se comunica com microcontrolador através do protocolo de comunicação I2C. Portanto, antes de iniciar a missão você precisa entender o que é isso. Assista ao vídeo “O que é I2C ? - Explorando o Arduino - Aula 10”, disponível no link: <https://www.youtube.com/watch?v=3cPDAQvu6ng>
- 2) Agora que você entende o protocolo I2C é hora de testar o sensor. Estude e teste o código apresentado no artigo “DISTANCE MEASUREMENT WITH VL53LOX”, disponível no link: <https://arduinoprojek.wordpress.com/distance-measurement-with-vl53lox/>
- 3) O robô que está sendo desenvolvido para a competição utiliza 2 sensores VL53L0X. A complexidade aumenta um pouco, então recomendamos que você leia o artigo “Usando Múltiplos Sensores de Distância Laser VL53L0X”, disponível em: <https://blog.smartk-its.com.br/usando-multiplos-sensores-de-distancia-laser-vl53l0x/>
- 4) Agora sim, você está pronto para iniciar a missão: Façam um código que envie pelo terminal serial a distância lida pelos 2 sensores a cada 1s. Mostrem para um professor que conseguiram cumprir a missão e expliquem o funcionamento do código. Após o professor anotar na planilha de acompanhamento (Checklist) que a tarefa foi cumprida, 1 integrante de cada equipe (apenas 1) deve fazer a entrega do arquivo do Arduino (*.ino) no Google Classroom. O aluno que entregar a atividade deve indicar (nos comentários) os nomes dos colegas que participaram ativamente do desenvolvimento da tarefa.

Módulo VL53L0X

Pinagem

Além de pinos de alimentação (VIN e GND), o módulo VL53L0X possui dois pinos dedicados a comunicação I2C (SCL e SDA), um pino de *reset* (XSHUT) e um pino de saída de dados (GPIO1) que pode ser utilizado para programar interrupções no microcontrolador ao qual o sensor está ligado.

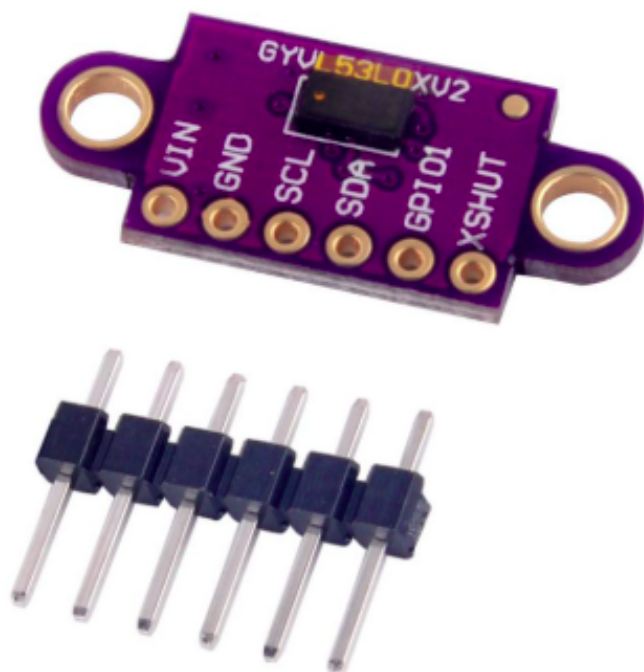


Imagem 1 — Módulo VL53L0X

Na tabela abaixo estão descritas as funcionalidades de cada pino.

VIN	Entrada de 2.6 V a 5.5 V. O regulador transformará a tensão em 2.8 V.
GND	Terra (0 V). O controlador PC deve utilizar o mesmo terra que a placa.
SDA	Terminal de dados I2C.
SCL	Terminal de clock I2C.
XSHUT	Shutdown input. Ativado em estado lógico baixo (LOW).
GPIO1	Output. Envia sinal quando a medição está completa.

Imagem 2 — Descrição dos pinos do VL53L0X